# ARDMS SYSTEM MANUAL

Advanced Rescue & Disaster Management System

Technical Operations Manual & System Workflows

## A. Project Overview & Abstract

The Advanced Rescue & Disaster Management System (ARDMS) is a complete digital solution designed to save lives during critical emergencies. It connects people in danger directly with nearby rescue teams and a central command center. By utilizing real-time GPS tracking, instant SOS alerts, and smart task grouping, ARDMS eliminates the confusion and delays typically seen during natural disasters or medical emergencies. This project provides a Web Admin Dashboard for operators, a Public App for victims to ask for help, and a Rescuer App for first responders to navigate exactly where they are needed.

## B. Introduction

When a disaster strikes - like a flood, earthquake, or severe medical emergency - every single second matters. Unfortunately, traditional phone calls to emergency numbers can get jammed, and explaining your exact location can be difficult when you are in a panic.

ARDMS solves this problem by using the technology already in our pockets: smartphones. With ARDMS, a person just has to press a single SOS button. Instantly, their exact GPS coordinates, the type of emergency, and their details are sent to a live Web Dashboard. An operator immediately sees this on a map and dispatches a registered Rescuer to that exact spot. The system acts as a digital bridge between those who need help and those who can provide it.

## C. System Architecture & Tech Stack

The system runs on a modern, real-time technology stack. It uses React Native for the mobile apps, React.js for the Web Dashboard, and a Node.js/SQLite backend for the database. Everything communicates instantly using WebSockets.

- Database Core: SQLite3 running in WAL (Write-Ahead Logging) mode to support concurrent transactions.

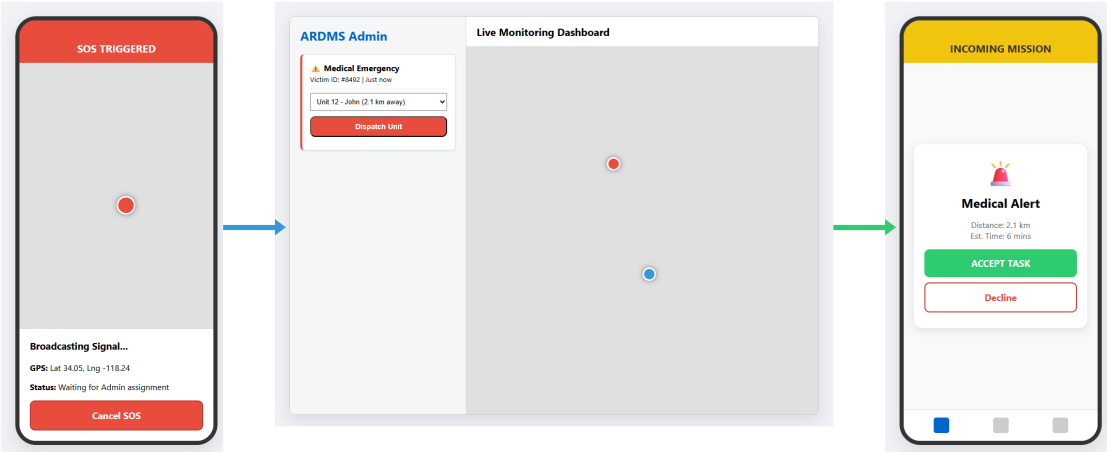
- Backend Server: Express.js REST API with integrated WebSockets for real-time telemetry streaming.

- Web Admin Center: HTML5 / Vanilla JS Single Page Dashboard styled with premium custom dark mode and glassmorphism, rendering dynamic incident markers via Leaflet.js.

- Mobile Applications: HTML5 Progressive Web App templates (Field Rescuer and Public SOS Apps) leveraging native GPS modules and offline AsyncStorage cache queues.

# D. Master Workflow Overview

Below is the complete overview diagram tracing the flow of an emergency. It traces the signal from the Public App triggering an SOS, to the Admin Dashboard dispatching it, directly to the Rescuer App accepting it.




1. Victim opens Public App and taps 'Medical Emergency'.
2. App waits 3 seconds, captures GPS, and POSTs data to Server.
3. Server saves to Database and alerts Web Admin.
4. Admin sees a flashing Red Alert on their map.
5. Admin clicks the alert, selects Rescuer 'John', and clicks 'Assign'.
6. John's Rescuer App rings. John clicks 'Accept'.
7. John follows the blue navigation line on his map to the victim.
8. Victim sees John's vehicle icon moving toward them on their screen.
9. John arrives, helps the victim, clicks 'Complete', and the alert turns green on the Admin dashboard.

# E. Master List of Operations

The ARDMS system operates through a set of interconnected workflows. Below is a structured list of every operation involved in the ecosystem, divided by platform and functional layer:

## 1. Triggering Requests (Public Citizen App Operations)

- Operation 1.1: Multi-Category Emergency Distress Broadcasting - Allows citizens to select specific emergency types (Medical, Flood, Fire, General Support) which triggers high-accuracy GPS capture, initiates a 3-second buffer cooldown timer, and uploads the payload.

- Operation 1.2: Real-Time Incident Telemetry Sync & Map Plotting - Listens to WebSocket broadcasts from the server containing the responder's moving coordinates to animate a vehicle icon.

- Operation 1.3: User-Initiated Distress SOS Resolution & Abort Flow - Safely cancels an active request, sweeps database tables, and returns the citizen to the main home UI.

- Operation 1.4: Offline Network Resilience & AsyncStorage Synchronization - Caches SOS payloads in local memory during poor coverage, bulk uploading them immediately on recovery.

## 2. Rescuer Operations (Rescuer Mobile Field App Operations)

- Operation 3.1: Instant Mission Incoming Handshake Alert & Wake Lock - Alerts responders via loud alarms, vibrations, device wake locks, and yellow flashing card overlays.

- Operation 3.2: Double-Lock Mission Acceptance & Real-Time GPS Activation - Updates mission command state to 'in-progress' in the database and triggers GPS tracking loops.

- Operation 3.3: Geographic Convex Hull Border Plotting & Zone Boundary Visualization - Renders a translucent purple outline around grouped tasks on the map with coordinate safety fallbacks.

- Operation 3.4: WebSocket Background Geolocation Telemetry Streaming - Automatically streams high-accuracy coordinates to `/api/telemetry` over WebSockets every few seconds.

- Operation 3.5: Physical Incident Resolution & Database State Finalization - Completes the task, resolving the active dashboard line and cascading states to all child items.

- Operation 3.6: Persistent Offline History Logging & Session Preservation - Archives completed/declined tasks in local memory, preserving credentials during selective cache washes.

## 3. Web System Operations (Web Admin Dashboard Operations)

- Operation 2.1: Multi-Channel Real-Time Incident Triage & Leaflet Mapping - Renders real-time incidents on Leaflet maps, featuring split columns for image preview lightboxes and inline WAV players.

- Operation 2.2: Proximity-Based Individual Dispatch & Mission Initialization - Tracks active rescuers, computes proximity lines, ranks rescuers by distance, and dispatches the task.

- Operation 2.3: Multi-Incident Tactical Grouping & Convex Hull Boundary Polygon Plotting - Bypasses auto-refresh race conditions using persistent `window.bulkSelectedIds` states to dispatch supply clusters.

- Operation 2.4: Active Route Polyline Memory Management & Map Cleanup - Auto-erases route

polylines upon completion or rejection to avoid visual map clutter.

- Operation 2.5: Dynamic In-Page Crew Re-allocation & Real-Time Redirection - Re-routes active responders mid-flight using in-page reassign modal pop-ups without page reloads.

## 4. Core Backend & Database Operations

- Operation 4.1: Database Write-Ahead Logging (WAL) Mode Initialization - Enables SQLite WAL mode on boot to allow concurrent, non-locking reads and writes during incident spikes.

- Operation 4.2: Dynamic Schema Migration & Verification - Validates database table parameters on startup, dynamically injecting new fields like `audio_url` without data loss.

- Operation 4.3: Base64 Audio SOS Decode and Directory Ingestion - Decodes base64 wave streams, saves binary audio files in folders, and links their path to the SQLite record.

- Operation 4.4: WebSocket Dynamic Multiroom Message Routing - Handles websocket connections, channels coordinates, and maps updates to correct room pipelines.

## 5. Network Deployment Operations

- Operation 5.1: Private Network-in-a-Box (NIB) Cellular Base Station Setup - Deploys private cellular SDR transceivers and programs private USIM cards for local coverage.

- Operation 5.2: Tailscale Private Mesh VPN Tunneling - Bypasses Carrier-Grade NAT (CGNAT) using virtual private networks to link remote devices over standard cell networks.

- Operation 5.3: Windows Defender Inbound Firewall Access Rule Setup - Creates PowerShell inbound firewall rules on port 3001 to prevent dynamic LAN connection drops.

# F. Detailed System Operations & Working Flows

This section describes each operation in detail, detailing the technical description, step-by-step workflow, high-tech flow diagrams, and user interface layouts.

## F.1 Public Mobile Support App Operations

The Public Mobile App allows citizens to broadcast emergency signals, track dispatched rescue units, and queue submissions during poor network conditions.

### Operation 1.1: Multi-Category Emergency Distress Broadcasting

Description: Allows citizens to select specific emergency types (Medical, Flood, Fire, General Support) which triggers a high-accuracy GPS capture and uploads a distress event. Initiates a 3-second countdown delay buffer to allow user cancellation in case of accidental clicks.

```
+-----------------------------------------------------------+
|  [Citizen Home UI] -> Select Emergency Type (Fire/Medical) |
|         |                                                 |
|  [3s Cooldown Timer] --(Cancel Tapped?)-> [Abort SOS]      |
|         |                                                 |
|  [Lock High-Accuracy GPS Coordinates]                     |
|         |                                                 |
|  [POST Distress Payload to /api/requests]                 |
|         |                                                 |
|  [WebSocket Broadcast to Web Admin] -> [Live Map UI]      |
+-----------------------------------------------------------+
```

Step-by-Step Instructions:

- Step 1: Open the ARDMS Public App on the mobile device.

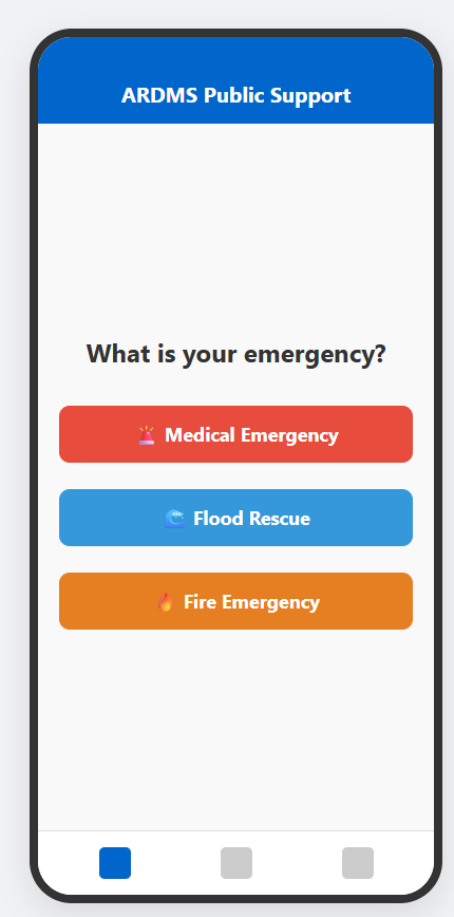
- Step 2: Identify the nature of the emergency on the home screen.

- Step 3: Tap the corresponding emergency button (e.g., 'Medical Emergency' or 'Flood Rescue').

- Step 4: A 3-second countdown banner (#selectionBufferCooldownBanner) appears. Wait or tap 'Cancel SOS' if triggered accidentally.

Visual UI Details (Home Screen Layout):

The mockup image below represents the emergency category selection home screen. It features a modern, accessible interface with warm slate styling. The primary buttons ('Medical Emergency', 'Flood Rescue', 'Fire Emergency') are large and high-contrast, ensuring ease of use for citizens under extreme panic or stress conditions.




Note: The category selection maps to urgency parameters on the backend (e.g. 'Critical' for Medical/Fire/Flood and 'Normal' for general support).

### Operation 1.2: Real-Time Incident Telemetry Sync & Map Plotting

Description: Connects the Public App to the server's WebSocket tracking rooms. The app continuously listens for location updates from the assigned responder and animates a custom map vehicle marker advancing toward the citizen.

```
+-----------------------------------------------------------+
|  [Establish Server WebSocket] -> Join Room 'rescue-room'  |
|         |                                                 |
|  [Listen for Rescuer Telemetry Broadcast]                 |
|         |                                                 |
|  [Extract Live Latitude & Longitude]                      |
|         |                                                 |
|  [Animate Rescuer Marker] -> [Update live ETA & Distance] |
+-----------------------------------------------------------+
```

Step-by-Step Instructions:

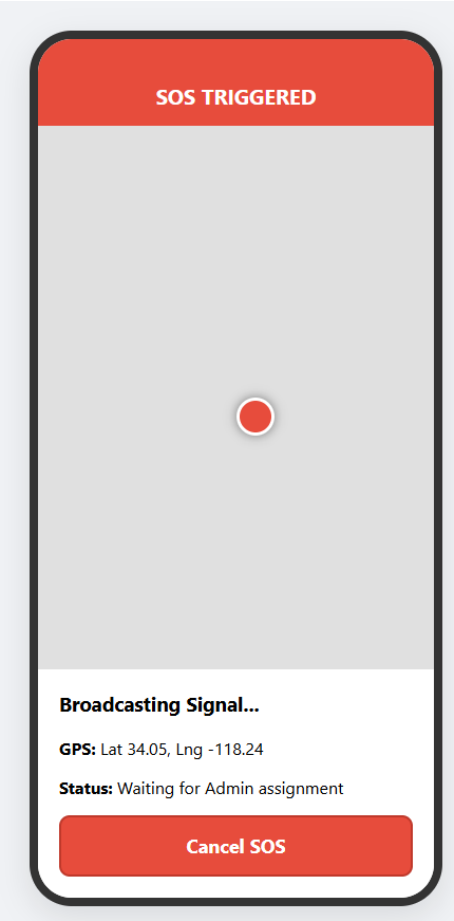
- Step 1: Once an SOS is active and assigned, the app automatically navigates to the Tracking page.

- Step 2: Verify the map loads showing your location and a blue vehicle marker.

- Step 3: Track the movement of the responder in real time along with the live distance readout.

Visual UI Details (Active Tracking Map UI):

The mockup image below shows the active tracking map. A red marker plots the citizen's static position, and a moving blue marker displays the rescuer's live telemetry coordinate in real-time, accompanied by estimated arrival metrics and an absolute 'Cancel SOS' button at the bottom.




Note: If the network drops during this operation, the app displays a prominent status indicator showing 'Offline - Reconnecting' while retaining the last cached coordinates of the responder.

### Operation 1.3: User-Initiated Distress SOS Resolution & Abort Flow

Description: Allows citizens to cancel their SOS alert if the situation resolves or the call was accidental, sending a delete command to the server and cleaning active command states.

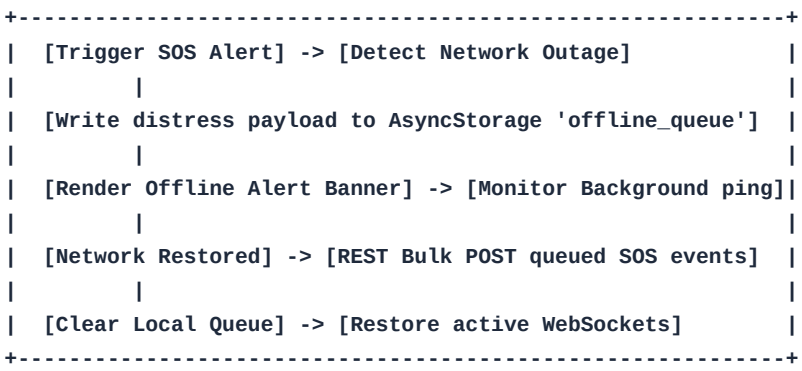
```
+-----------------------------------------------------------+
|  [User clicks 'Cancel SOS' Button]                        |
|         |                                                 |
|  [REST DELETE /api/requests/:id/cancel]                   |
|         |                                                 |
|  [Database Sweeps State] -> [WS Broadcast to Web Admin]   |
|         |                                                 |
|  [Clear active session tokens] -> [Redirect to Home UI]   |
+-----------------------------------------------------------+
```

Step-by-Step Instructions:

- Step 1: Tap the 'Cancel SOS' button at the bottom of the tracking screen.
- Step 2: Confirm the cancellation prompt on the pop-up modal.
- Step 3: The app immediately returns to the home page, clearing active alert flags.

### Operation 1.4: Offline Network Resilience & AsyncStorage Synchronization

Description: Implements local offline caches. When the system detects a network drop, it saves the SOS report in local device memory and dynamically schedules queue pushes upon internet recovery, preventing data loss.




Step-by-Step Instructions:

- Step 1: Trigger an emergency alert while offline.
- Step 2: A yellow notification bar will alert you that your request is safely stored in local cache.
- Step 3: When cellular network is re-established, the app automatically synchronizes and updates the database.

## F.2 Web Admin Dashboard Operations

The Command Center Web Dashboard gives dispatcher teams comprehensive mapping controls, tactical group building tools, and real-time crew assignment utilities.

### Operation 2.1: Multi-Channel Real-Time Incident Triage & Leaflet Mapping

Description: Integrates raw WebSocket packets with Leaflet.js map layers, showing flashing red incident markers for urgent requests, while rendering separate media columns for IMAGE attachments and inline play/download AUDIO WAV players.

```
+-----------------------------------------------------------+
|  [WS Connection Ready] -> [Capture Incoming SOS Alert]     |
|         |                                                 |
|  [Query DB Details] -> [Format & Parse Media Uploads]      |
|         |                                                 |
|  [Plot Flashing Red Leaflet Marker] -> [Render split media]|
|         |                                                 |
|  [Play Sound Indicator] -> [Populate Triage Grid Columns]  |
+-----------------------------------------------------------+
```

Step-by-Step Instructions:

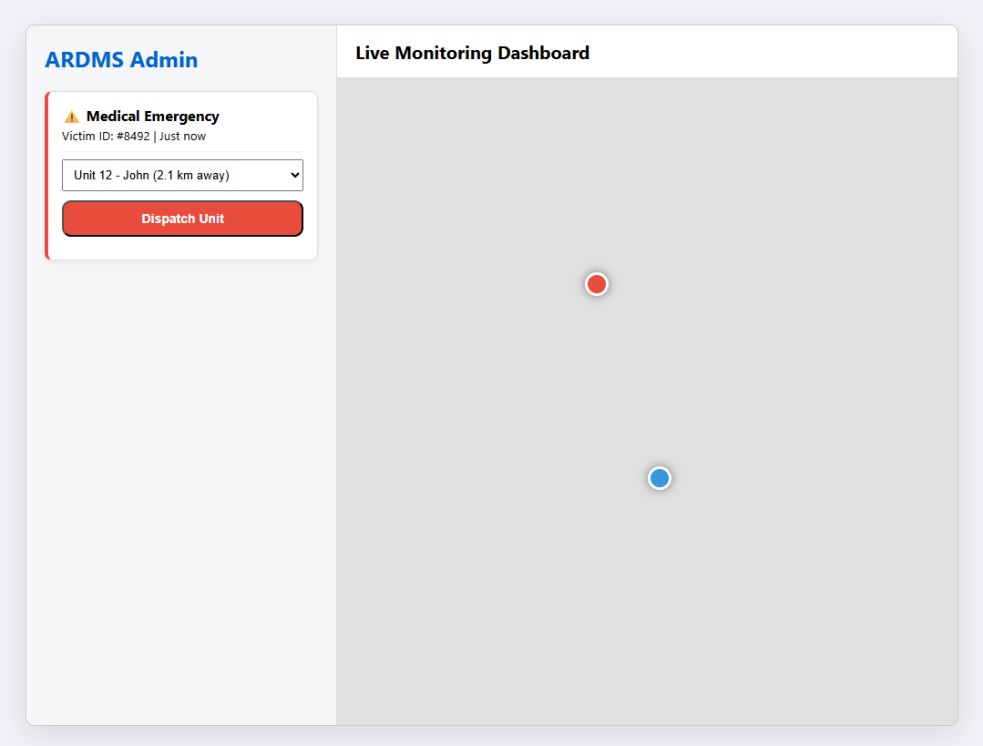
- Step 1: Open the ARDMS Command Center portal in a browser.

- Step 2: Review the map grid displaying active rescue markers.

- Step 3: Access incoming media reports directly via the split columns (thumbnails & inline WAV player).

Visual UI Details (Command Dashboard & Proximity Dispatch):

The mockup image below represents the Web Admin Command Center dispatch panel. The central map displays active incidents (red markers) and responder vectors (blue markers). Clicking a marker opens the sidebar dispatch UI, allowing operators to match the task with a proximity-ranked rescuer.




Note: The dispatch dropdown lists rescuers in real-time, sorted by proximity to ensure fast arrivals.

### Operation 2.2: Proximity-Based Individual Dispatch & Mission Initialization

Description: Maps individual rescue tasks. When an incident is selected, the system queries the coordinates of all active online rescuers, calculates direct GPS line distance, ranks them, and dispatches the mission to the closest available officer.

```
+-----------------------------------------------------------+
|  [Operator clicks Pin] -> [Query all active rescuers]     |
|         |                                                  |
|  [Calculate Proximity Metric (GPS coords distance)]       |
|         |                                                  |
|  [Sort Rescuer dropdown dynamically by shortest distance]  |
|         |                                                  |
|  [Click Dispatch] -> [POST /api/commands] -> [WS to Mobile]|
+-----------------------------------------------------------+
```

Step-by-Step Instructions:

- Step 1: Select an active emergency marker on the Leaflet map.
- Step 2: In the dispatch panel, review the proximity-sorted list of nearby rescuers.
- Step 3: Select the optimal unit and click 'Dispatch Unit' to send the mission details.

### Operation 2.3: Multi-Incident Tactical Grouping & Convex Hull Boundary Polygon Plotting

Description: Bundles multiple incident markers in a disaster zone into a single group. Stores checkboxes inside `window.bulkSelectedIds` globally so background auto-refresh runs do not clear the selection. Computes a convex hull bounding box around selected markers and dispatches a single heavy-duty squad with a unified multi-waypoint zone route.

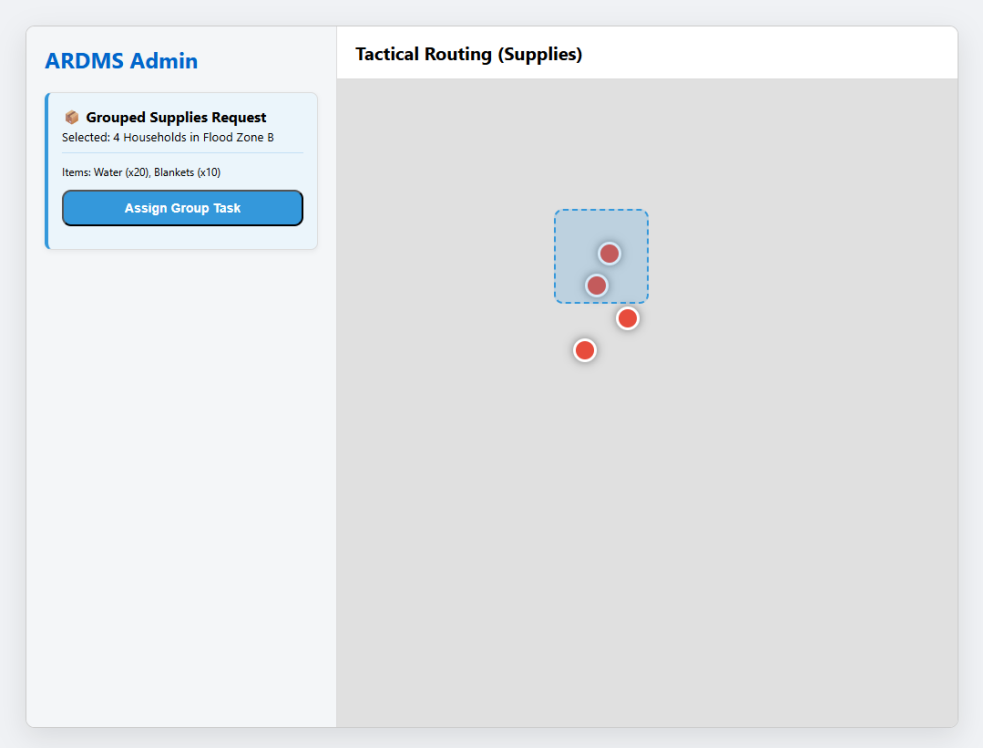
```
+-----------------------------------------------------------+
|  [Operator checks multiple emergency checkboxes]          |
|         |                                                 |
|  [Cache selections inside window.bulkSelectedIds]         |
|         |                                                 |
|  [Open Bulk Group View] -> [Compute Convex Hull Polygon]  |
|         |                                                 |
|  [Specify supply list] -> [Select heavy squad team]       |
|         |                                                 |
|  [POST /api/commands/group] -> [Draw boundary hull on map] |
+-----------------------------------------------------------+
```

Step-by-Step Instructions:

- Step 1: Check the selection boxes next to multiple emergencies in the triage board.
- Step 2: Click 'Group & Assign' to create a Tactical Cluster.
- Step 3: Review the computed convex hull polygon bounding box on the selection map.
- Step 4: Specify necessary supplies (e.g. water boxes, emergency blankets).
- Step 5: Select the target crew and click 'CONFIRM & INITIALIZE GROUP MISSION'.

Visual UI Details (Tactical Convex Hull Grouping):

The mockup image below displays the tactical group creation view. A bounding box polygon is computed and plotted around a cluster of multiple flood victims in Sector B. The sidebar transitions to the 'Grouped Supplies Request' panel for dispatching a single heavy-duty responder.

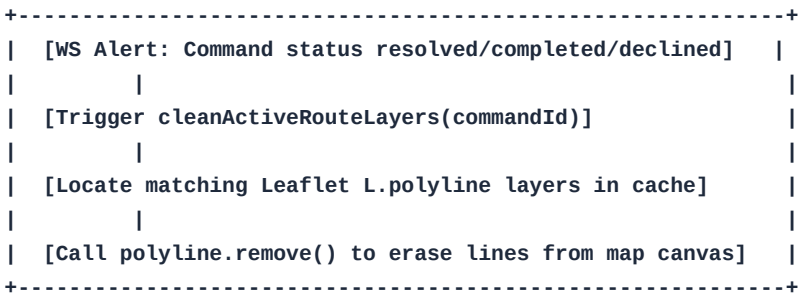



Note: This bulk method avoids redundant dispatches and reduces radio/data traffic for multiple incidents in close proximity.

### Operation 2.4: Active Route Polyline Memory Management & Map Cleanup

Description: Manages map memory. When a task is completed, resolved, or declined, the system

calls `cleanActiveRouteLayers()` to identify and remove matching polyline route graphics from the map canvas, keeping the view clean.




Step-by-Step Instructions:

- Step 1: The map shows blue dashed lines indicating active responder paths.

- Step 2: When an officer marks a task complete or cancels, the line immediately vanishes from the screen.

### Operation 2.5: Dynamic In-Page Crew Re-allocation & Real-Time Redirection

Description: Allows operators to redirect a squad mid-flight. Clicking 'Re-assign' accesses the command ID and triggers the re-assignment modal, sending a redirection handshake to both the original and new mobile apps.


Step-by-Step Instructions:

- Step 1: Click the 'RE-ASSIGN' button next to an active command.

- Step 2: Choose the new responder and click 'Update Assignment'.

- Step 3: Confirm the redirection update; the original task is reassigned without reloading the page.

## F.3 Rescuer Mobile Field App Operations

The Rescuer Mobile App provides emergency responders with continuous routing instructions, incident details, and direct synchronization with the Command Center.

### Operation 3.1: Instant Mission Incoming Handshake Alert & Wake Lock

Description: Triggers high-priority visual and audio alerts when a new mission is dispatched, sounding a loud ringer, pulsing the motor, and waking the device screen so rescuers are notified instantly.

```
+-----------------------------------------------------------+
|  [WS Command broadcast parsed by Rescuer App client]      |
|         |                                                 |
|  [Activate Android Device Wake Lock] -> [Loop loud alarm] |
|         |                                                 |
|  [Flash high-visibility yellow 'INCOMING MISSION' alert]  |
|         |                                                 |
|  [Render mission distance, ETA, and priority level]       |
+-----------------------------------------------------------+
```

Step-by-Step Instructions:

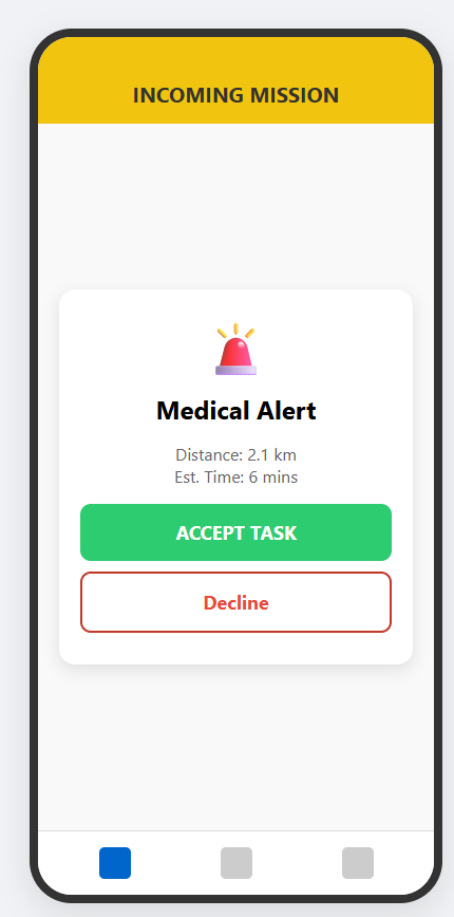
- Step 1: Keep the Rescuer App running.

- Step 2: When a task is dispatched, the phone will ring loudly and display the incoming mission screen.

- Step 3: Review the distance, estimated time, and incident type.

Visual UI Details (Incoming Mission Interface):

The mockup image below represents the high-priority alert flashed on the rescuer's smartphone. The entire card flashes yellow, looping a loud warning sound. The rescuer can tap the large green 'ACCEPT TASK' button or decline, routing the task back.




Note: The wake lock prevents the screen from turning off, ensuring notifications are visible even when the device is locked.

### Operation 3.2: Double-Lock Mission Acceptance & Real-Time GPS Activation

Description: Locks the rescuer to the task by posting an accept state to the database, switching the app to navigation mode and activating active GPS updates.

```
+-------------------------------------------------------------+
|  [Rescuer taps green 'ACCEPT TASK' button]                  |
|         |                                                    |
|  [POST /api/commands/:id/accept] -> Update status in DB     |
|         |                                                    |
|  [Retrieve target coordinates] -> [Initialize GPS tracking]|
|         |                                                    |
|  [Lock active accept states] -> [Navigate to Tracking UI]   |
+-------------------------------------------------------------+
```

Step-by-Step Instructions:

- Step 1: Tap the green 'ACCEPT TASK' button on the alert card.
- Step 2: The app will update its status and transition to the map view.

### Operation 3.3: Geographic Convex Hull Border Plotting & Zone Boundary Visualization

Description: Visualizes group mission zones. The app reads the `missions` list of coordinates and

draws a translucent boundary. To prevent crashes or zooming to `[0, 0]` when center coordinates are missing, `startMission()` falls back to using the coordinates of the first mission.

```
+-----------------------------------------------------------+
|  [Receive group command payload]                          |
|         |                                                 |
|  - Center Coordinates Missing? -> Fallback to 1st mission |
|         |                                                 |
|  [Calculate Convex Hull envelope surrounding all tasks]   |
|         |                                                 |
|  [Draw purple border overlay] -> [Plot markers for childs] |
+-----------------------------------------------------------+
```

Step-by-Step Instructions:

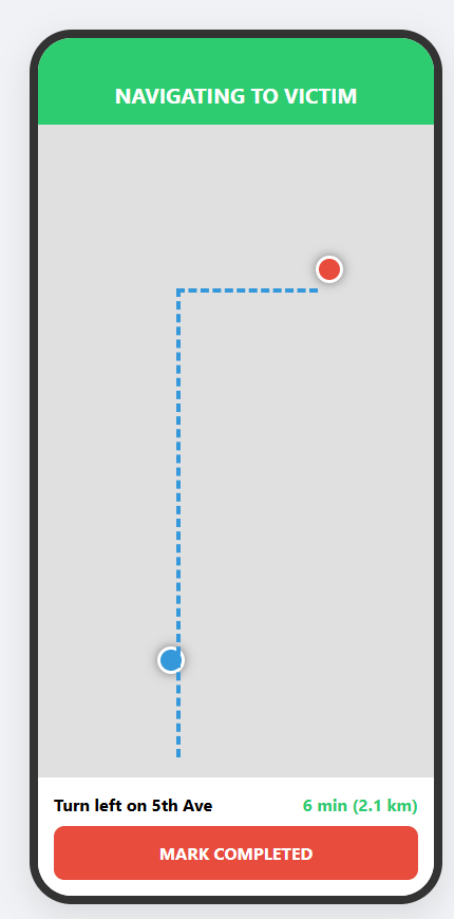
- Step 1: Accept a grouped tactical mission.

- Step 2: Observe the map zoom to the region, plotting a translucent purple outline around all selected victims.

- Step 3: Tap any victim pin within the boundary to view their specific needs.

Visual UI Details (Active Incident Navigation View):

The mockup image below represents the active tracking map on the rescuer app. A blue path outlines the direct route to the target location. Proximity distance and turn instructions are visible in the overlay, and a 'MARK COMPLETED' button is placed at the bottom.




Note: Geolocation parameters update coordinates in the background via REST, ensuring data

consistency.

### Operation 3.4: WebSocket Background Geolocation Telemetry Streaming

Description: Streams location updates. When a mission is active, a background thread monitors GPS changes and streams them to the server over WebSockets every few seconds, updating both the Admin and Citizen maps.

```
+-----------------------------------------------------------+
|  [Mission In-Progress] -> [Start background location thread|
|         |                                                 |
|  [Capture continuous device GPS coords (lat, lng)]        |
|         |                                                 |
|  [Send telemetry packet to /api/telemetry over WS]        |
|         |                                                 |
|  [Command center and Citizen client update live vectors]  |
+-----------------------------------------------------------+
```

Step-by-Step Instructions:

- Step 1: Proceed along the route towards the incident location.

- Step 2: The app automatically streams your current position to the server in the background.

### Operation 3.5: Physical Incident Resolution & Database State Finalization

Description: Completes the task. Tapping 'MARK COMPLETED' sends a complete command to the server, which sets the status to completed in the database, cascades the status to any child requests in a group, and archives the event.

```
+-----------------------------------------------------------+
|  [Rescuer arrives at scene & resolves the rescue incident] |
|         |                                                 |
|  [Taps 'MARK COMPLETED' button]                           |
|         |                                                 |
|  [POST /api/commands/:id/complete]                        |
|         |                                                 |
|  [Server cascades state to child requests]                |
|         |                                                 |
|  [Clear routing layers] -> [Revert back to main feed UI]  |
+-----------------------------------------------------------+
```

Step-by-Step Instructions:

- Step 1: Upon physical arrival and successful resolution of the rescue task, tap 'MARK COMPLETED'.

- Step 2: Verify that your map is cleared of the route, returning you to the active task feed.

### Operation 3.6: Persistent Offline History Logging & Session Preservation

Description: Caches history logs. The app queries specialized history endpoints to show a list of active, completed, and declined tasks. The clear cache function selectively removes history logs without affecting active session tokens.

```
+-----------------------------------------------------------+
|  [Navigate to History Tab] -> [Fetch complete user history]|
```

```
|         |                                                        |
|  [Render all-status logs (completed, declined, active)]    |
|         |                                                        |
|  [Mirror log archive to AsyncStorage for offline backup]   |
|         |                                                        |
|  [Selective cache clear sweeps history, preserves session] |
+------------------------------------------------------------+
```

Step-by-Step Instructions:

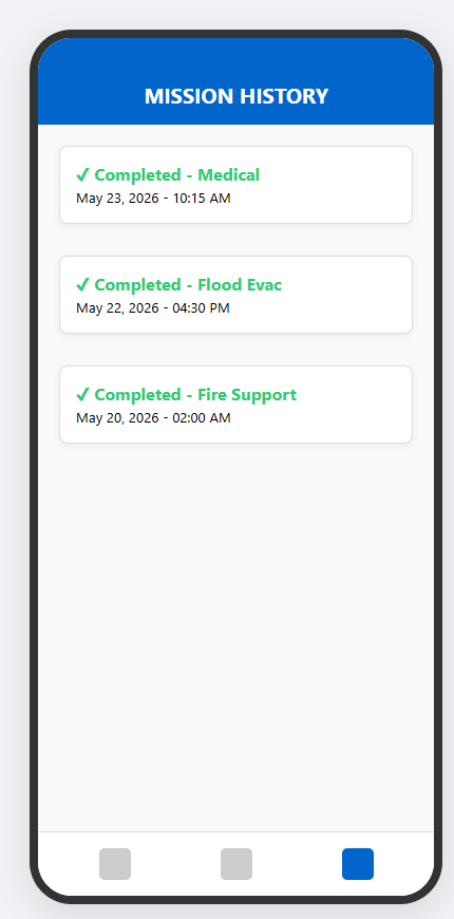
- Step 1: Open the Rescuer App and select the 'History' tab on the navigation bar.

- Step 2: Scroll through the history feed to review your completed assignments.

- Step 3: Tap 'Clear History Cache' in settings to clean data logs while remaining logged in.

Visual UI Details (Rescuer History Panel):

The mockup image below represents the mission history logs screen. Chronologically ordered cards display previous missions. Tapping a card displays complete dispatch logs, response times, and final resolutions.




Note: The offline cache ensures rescuers can access these logs even in dead network zones.

## G. Task Lifecycle Workflow

Every emergency requested by the Public App becomes a 'Task' in the ARDMS system. A Task follows a strict operational lifecycle to ensure no emergency is left unresolved.

- State 1: PENDING - The moment an SOS is triggered, a database record is created. It appears as a flashing red alert on the Admin Dashboard.

- State 2: DISPATCHED - Once the Admin selects a Rescuer and clicks 'Dispatch', the payload is pushed to the Rescuer's device.

- State 3: IN-PROGRESS - When the Rescuer hits 'ACCEPT TASK', the status changes. The routing line is drawn, and the ETA begins counting down.

- State 4: COMPLETED - Upon arriving at the scene and resolving the crisis, the Rescuer presses 'MARK COMPLETED'. The task is cleared from the active map and securely logged into the History Database.

## H. Dynamic Map & Routing Workflow

The mapping system is the visual core of ARDMS, providing real-time geographical awareness.

- 1. Initial Geolocation: The Public App uses native GPS modules to lock onto high-accuracy coordinates (Latitude/Longitude) before transmitting the SOS payload.

- 2. Admin Plotting: The Node.js server receives these coordinates and instantly plots a Red Emergency Marker on the Web Dashboard's live map.

- 3. Routing Calculation: Upon task assignment, the Rescuer App queries routing APIs to draw a blue tactical path linking their current location to the victim's location.

- 4. Live Telemetry Sync: As the Rescuer physically moves, their phone pulses updated coordinates every few seconds via WebSockets. The server reflects this movement, visibly animating the Rescuer's icon advancing along the route on both the Admin's screen and the Victim's screen.